

Car Price Predictor - Technical Documentation

Generated: July 06, 2026

1. Project Overview

The Car Price Predictor is a Streamlit web application that estimates the selling price of a used car in Ghanaian Cedis (GHS) based on user inputs. The app accepts five parameters -- brand, year, mileage, fuel type, and transmission -- and returns a predicted price using a machine learning model trained on historical car sale data.

2. Type of Learning: Supervised Learning

This project employs Supervised Learning: a branch of machine learning where the model learns from labeled training data. Each training example consists of input features (car attributes) paired with a known target value (the actual selling price). The model discovers patterns mapping inputs to outputs, then applies those patterns to make predictions on unseen data.

Within supervised learning, this is a Regression task (not classification) because the target variable -- car price -- is a continuous numerical value rather than a discrete category.

- Supervised Learning -- The training data includes both features and correct answers (prices).
- Regression -- The output is a continuous number (price in GHS), not a class label.
- Batch Learning -- The model is trained once on the full dataset offline (not incrementally).
- Model-Based Learning -- We fit a mathematical model to the data, rather than memorizing examples.

3. Algorithm: Linear Regression

Linear Regression is one of the simplest and most widely used regression algorithms. It models the relationship between the input features and the target variable as a linear equation:

$$\text{Price} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + \dots + w_n \cdot x_n$$

Where:

- w_0 is the intercept (bias term).
- $w_1 \dots w_n$ are the learned coefficients (weights) for each feature.
- $x_1 \dots x_n$ are the input feature values (year, mileage, brand encodings, etc.).

The model learns these coefficients during training by minimizing the Mean Squared Error (MSE) between predicted and actual prices across all training examples. This is done using the Ordinary Least Squares method, which finds the optimal weights analytically.

Mathematical formulation:

$$\text{MSE} = (1/n) * \text{SUM} (y_{\text{true}} - y_{\text{pred}})^2$$

$$\text{Optimal } w = (X^T * X)^{-1} * X^T * y$$

4. Why We Chose Linear Regression

- Simplicity and Speed -- Training completes in milliseconds, even on large datasets. No hyperparameter tuning required, making it ideal for a quick demo application.
- Interpretability -- Each coefficient directly tells us how much the price changes per unit change in that feature. For example, the year coefficient tells us how much value a car gains per additional model year.
- Strong Baselines with Nearly-Linear Features -- Car mileage has an approximately linear negative relationship with price (more km = lower price). Model year has an approximately linear positive relationship (newer = higher price). With one-hot encoded categorical features (brand, fuel, transmission), the linear model can capture brand-specific premiums effectively.
- Low Data Requirements -- Unlike tree-based or neural methods, linear regression performs well even with limited data and is less prone to overfitting when the number of samples is modest.
- Well-Understood Assumptions -- The assumptions (linearity, independence, homoscedasticity, normality of errors) are well-documented and can be validated, making debugging straightforward.

5. Code Walkthrough

5.1 Imports and Page Setup

```
import streamlit as st
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

st.set_page_config(page_title="Car Price Predictor",
                    layout="centered",
                    initial_sidebar_state="collapsed")
```

Streamlit provides the web UI framework. Pandas handles data manipulation. Scikit-learn provides the train/test split and the LinearRegression estimator. The page config sets a clean, centered layout.

5.2 Data Loading and Preprocessing

```
@st.cache_data
def load_and_train():
    try:
        URL = ("https://raw.githubusercontent.com/"
               "YBIFoundation/Dataset/main/Car%20Price.csv")
        df = pd.read_csv(URL)
    except Exception:
        import os
        df = pd.read_csv(os.path.join(
            os.path.dirname(__file__), "car_price.csv"))
```

The `@st.cache_data` decorator caches the function result so the dataset is loaded and the model is trained

only once (not on every user interaction). A try/except block attempts the remote URL first and falls back to a local CSV if the network is unavailable.

5.3 Feature Engineering

```
df = df[["Brand", "Year", "KM_Driven", "Fuel",
        "Transmission", "Selling_Price"]]
df.columns = ["brand", "year", "mileage_km",
              "fuel_type", "transmission", "price"]
df = df[(df["price"] > 50000) & (df["price"] < 10000000)]

df_encoded = pd.get_dummies(
    df, columns=["brand", "fuel_type", "transmission"],
    drop_first=True)
```

The six most relevant columns are selected and renamed. Prices below 50,000 and above 10,000,000 are filtered as outliers. One-hot encoding (pd.get_dummies) converts categorical features into binary indicator columns. drop_first=True avoids the dummy variable trap by dropping one category per feature as the reference baseline.

5.4 Training

```
X = df_encoded.drop("price", axis=1)
y = df_encoded["price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
return model, X_train.columns.tolist(), df
```

Features (X) are separated from the target (y). The data is split 80/20 into training and test sets with a fixed random seed for reproducibility. The LinearRegression model is instantiated and trained via .fit(). The function returns the trained model, the list of feature column names (needed later to build the prediction input row), and the raw DataFrame (needed for the UI dropdowns).

5.5 User Interface

```
model, feature_cols, df = load_and_train()
st.title("Car Price Predictor")

brand = st.selectbox("Brand", sorted(df["brand"].unique()))
fuel_type = st.selectbox("Fuel Type", sorted(df["fuel_type"].unique()))
transmission = st.selectbox("Transmission", sorted(df["transmission"].unique()))
year = st.selectbox("Year", list(range(2005, 2025)))
mileage_km = st.selectbox("Mileage (km)",
                          [10000, 30000, 50000, 70000, 100000, 150000, 200000])
```

The trained model and data are loaded once (cached). Five input widgets collect the user car specifications. Dropdown options are populated dynamically from the dataset unique values.

5.6 Prediction

```
if st.button("Predict Price"):
```

```
row = {col: 0 for col in feature_cols}
row["year"] = year
row["mileage_km"] = mileage_km
for prefix, val in [("brand_", brand),
                    ("fuel_type_", fuel_type),
                    ("transmission_", transmission)]:
    col = prefix + val
    if col in row:
        row[col] = 1

price = model.predict(pd.DataFrame([row]))[0]
st.success(f"### GHS {price:,.0f}")
```

When Predict is clicked, an empty dictionary is initialized with zeros for all feature columns (matching the training feature set). Numeric values (year, mileage) are set directly. Categorical values activate the matching one-hot encoded column (e.g., "brand_Toyota"). The resulting row is passed to `model.predict()`, and the predicted price is displayed formatted with thousand separators in GHS.

6. Limitations and Future Improvements

- Linear Assumption -- Linear regression cannot capture non-linear relationships or feature interactions (e.g., the combined effect of brand and age on price). A tree-based model like RandomForest or XGBoost could improve accuracy.
- No Feature Scaling -- While linear regression is scale-invariant for coefficient estimation, unscaled features can affect regularization if L1/L2 penalty were added in the future.
- Limited Features -- Only 5 features are used. Including more (owner count, service history, accident record, location) would improve prediction quality.
- No Validation -- The current split only uses train/test. Cross-validation would give a more reliable estimate of model performance.
- No Model Metrics Displayed -- The app does not show R-squared, MAE, or RMSE to the user, so prediction confidence is unknown.

7. Summary

This application demonstrates a complete supervised regression pipeline: data loading, cleaning, one-hot encoding, train/test splitting, linear regression training, and real-time inference via a Streamlit UI. Linear regression was chosen for its simplicity, speed, interpretability, and strong performance on nearly-linear feature relationships typical in used car pricing.